

Action-Conditioned Predictive Diagnostics for JEPA World Models

Author names to be supplied for arXiv v1

July 2026

Abstract

Latent predictive world models avoid reconstructing every pixel, but latent prediction alone does not show whether state-preserving visual changes remain harmless after learned dynamics and planning. We introduce Action-Conditioned Predictive Consistency (ACPC), a frozen-checkpoint diagnostic that compares the predicted latent trajectories produced by nominal and visually perturbed views of the same trajectory under shared actions. Because a constant representation can make this disagreement vanish, we pair ACPC with a state-coordinate proxy test for different-state separation. Exact samplewise inequalities relate ACPC to error differences against a shared observed future and to movement in squared latent goal cost. Across LeWM checkpoints on four control tasks, recorded-action eight-step ACPC predicts perturbation-induced future-error change with $51.3 \pm 3.5\%$ lower held-out MAE than the best same-horizon control without the recorded actions. Across fixed-pool and adaptive-CEM evaluations, candidate-conditioned five-step ACPC also improves held-out prediction of cost movement and model-cost decision regret. Source-task thresholds retain checkpoint-screening accuracy on held-out tasks, while score changes remain rank-correlated with planning-success changes at the tested blur and resize severities. We repeat the analysis on PLDM using the same procedure, with numerical thresholds recalibrated for that model family.

Keywords: visual world models; predictive stability; action-conditioned diagnostics; calibration transfer; visual corruption.

Code and data release: <https://github.com/Anguo-star/lewm-acpc-diagnostics>

1 Introduction

World models support control by predicting how candidate actions change the environment [11, 12]. Joint-embedding predictive architectures (JEPAs) move this prediction problem from pixels to learned representations [16, 1, 3]. By avoiding pixel reconstruction, they can reduce pressure to model nuisance detail [28, 17]. This is appealing for visual control, where lighting, texture, or image quality may change even when the physical situation does not. Latent prediction alone, however, does not establish that a trained world model will produce stable action-conditioned predictions under such changes.

For control, whether a visual difference is irrelevant depends on the state and the actions being considered. A background texture can be ignored, whereas a contact pose, doorway state, or object-goal relation may determine which action is useful. Encoder invariance cannot resolve this distinction on its own because a planner does not act directly on the encoder output. It rolls the representation forward under candidate actions and ranks the resulting predictions with a cost. A small encoder discrepancy may grow through this computation, whereas a larger one may contract before it affects the cost. This motivates the central question: after the same action sequence is applied, do nominal and perturbed views of one trajectory still produce similar predicted futures?

Low rollout disagreement alone is not enough. A model that maps every observation to a constant representation would make all paired rollouts identical while retaining no state information for control. A useful diagnostic must therefore test two properties together: same-state visual views should remain close throughout the predicted rollout, while pairs identified as different by task state coordinates should remain separated. The latter test is a deliberately limited collapse guard. It assesses only the supplied coordinate labels and does not certify every semantic or action-relevant distinction.

We introduce Action-Conditioned Predictive Consistency (ACPC) to quantify same-state rollout stability. ACPC propagates nominal and visually perturbed histories of one trajectory under a shared action sequence

and measures the distance between their predicted latent trajectories; the pairwise score itself requires neither a realized future nor a task outcome. Evaluating both predictions against the same logged future gives an exact bound on the difference between their errors. For LeWM’s squared latent goal cost, a second bound limits candidate-cost movement and yields fixed-pool stability certificates when combined with the clean candidate margins. At the checkpoint level, Action-Conditioned Trajectory Radius (ATR) summarizes high-quantile normalized disagreement, while Selective Margin Pass Rate (SMPR) checks whether coordinate-labeled different-state pairs remain outside the raw same-state radius by a prescribed margin. A calibrated score combining ATR and SMPR screens frozen checkpoints for visual predictive sensitivity. ACPC is an evaluation tool, not a new training objective.

We evaluate this diagnostic on LeWM [19] across TwoRoom, PushT, Reacher, and Cube, using Gaussian observation noise for calibration. Using recorded-action eight-step ACPC lowers held-out MAE for predicting the change in future error by $51.3 \pm 3.5\%$ relative to the strongest same-horizon control that destroys the recorded action sequence. Candidate-conditioned five-step ACPC consistently improves prediction of adaptive-CEM model-cost decision regret; its improvement for maximum fixed-pool cost movement is positive on three of the four tasks. Aggregated over all 14 directional partitions, balanced accuracy is 0.855 with one source task and 0.900 with either two or three. Gaussian-calibrated score changes also track planning-success changes at one tested severity for each of blur and resize. The same analysis can be instantiated on PLDM, but its numerical thresholds require model-family-specific recalibration.

The contributions are:

1. **A paired-rollout diagnostic.** ACPC measures how same-state visual changes propagate through action-conditioned predictions, ATR summarizes the checkpoint-level radius, and SMPR provides a state-coordinate guard against constant-representation collapse.
2. **Exact links to prediction and planning quantities.** Samplewise inequalities connect rollout disagreement to common-future error drift and squared latent goal-cost movement.
3. **Controlled evidence beyond the calibration tasks.** Experiments test predictive relevance and calibration transfer across tasks, fixed visual shifts, and a second model family.

2 Related Work

Prior work addresses three complementary parts of this problem: learning latent predictive representations, preserving control-relevant distinctions under visual shifts, and evaluating learned dynamics.

Latent predictive world models. DreamerV3 and TD-MPC2 illustrate how learned dynamics can support control by predicting the consequences of candidate actions [11, 12]. JEPAs replace pixel reconstruction with prediction in representation space, first for images and then for video [16, 1, 3, 2]. LeWM instantiates this idea as an end-to-end JEPA world model stabilized by SIGReg [19, 7], while PLDM provides a different joint-embedding dynamics design [25, 26, 18]. Analyses of latent prediction explain why it can suppress nuisance or noisy features [28, 17, 14]; Seq-JEPA studies the related balance between invariant and equivariant prediction [9]. These results motivate latent predictive learning, but our focus is checkpoint evaluation: whether the discrepancy between two visual views of the same trajectory contracts or grows as the predictor rolls them forward under shared actions.

Robust visual control and selective invariance. DrQ, DrQ-v2, and SODA improve visual control through training-time augmentation [15, 32, 13]. ViGMO and VIBR learn invariances in latent or value space [20, 6], whereas ReOI modifies observations at test time to reduce distractor sensitivity in visual model-predictive control [5]. These methods seek to train or repair a more robust controller. We instead ask how to audit visual sensitivity in an already trained predictive model.

Control-aware representation learning also explains why robustness cannot mean removing every difference. Bisimulation-based methods preserve distinctions in rewards and transitions [8, 33, 24, 27], while value-equivalent and value-aware models preserve distinctions that matter to downstream value or planning [10, 29]. This motivates the two-sided structure of our diagnostic: same-state visual views should agree after prediction, but low disagreement is considered meaningful only when pairs marked as different by task state coordinates remain separated. Our state-coordinate test is narrower than a learned bisimulation or semantic criterion; its role is to expose the constant-representation failure mode.

Diagnostics of learned dynamics. Recent work audits whether learned dynamics retain action or temporal structure using inverse prediction, latent action decoding, cycle consistency, group-action tests, or compatibility with generated futures [4, 34, 23, 21, 30]. Related approaches place action-conditioned consistency inside the training objective [31] or diagnose kinematic failures in long-horizon rollouts [22]. Together, these studies motivate evaluating the predictor directly rather than inferring its behavior from encoder geometry alone. ACPC asks a complementary paired-view question: under one shared action sequence, how much does a state-preserving visual change move the predicted rollout? We then test how that displacement relates to the change in prediction error against a common observed future, fixed-pool candidate-cost movement, and adaptive-CEM decision regret. For checkpoint screening, ATR and SMPR combine the rollout radius with the limited state-coordinate proxy guard; their numerical calibration is performed within each model family.

3 Action-Conditioned Predictive Consistency as a Diagnostic

A visual perturbation can change pixels without changing the underlying control state. It nevertheless displaces the encoded history, and learned dynamics can amplify, rotate, or contract that displacement before a planner reads out a cost. Encoder invariance alone therefore leaves two questions unanswered: how far do the predicted futures separate under the same actions, and can that separation change a planning decision?

The desired diagnostic has two sides. Two visual views of the same state should remain inside a small action-conditioned rollout tube. At the same time, states that differ under a declared task proxy should remain outside that tube. The first side supplies a predictive radius; the second prevents a low radius from being mistaken for robustness when the representation has collapsed. We first define the radius and connect it to prediction error and planner decisions, then introduce the complementary task-proxy margin and checkpoint score.

3.1 Paired rollout radius

Let h be a clean observation-action history and $\tilde{h}$ a visually perturbed view of the same underlying state trajectory. A frozen encoder gives $z = E_\theta(h)$ and $\tilde{z} = E_\theta(\tilde{h})$. Both branches receive the same action sequence $\mathbf{a} = (a_0, \dots, a_{H-1})$. For its length- k prefix, define

$$\hat{z}_k = F_\theta^k(z, \mathbf{a}_{0:k-1}), \quad \hat{\tilde{z}}_k = F_\theta^k(\tilde{z}, \mathbf{a}_{0:k-1}). \quad (1)$$

Here F_θ is the action-conditioned predictor and H is the number of autoregressive future steps. The map Π selects the space in which rollout distances and planner costs are evaluated. In our experiments this is the checkpoint’s normalized inference-cost embedding space, so Π is the identity on the stored projected embeddings. With nonnegative weights $\sum_{k=1}^H \alpha_k = 1$, stack the whole rollout as

$$\bar{G}_\mathbf{a}(z) = [\sqrt{\alpha_1}\Pi(\hat{z}_1)^\top \quad \dots \quad \sqrt{\alpha_H}\Pi(\hat{z}_H)^\top]^\top. \quad (2)$$

Action-Conditioned Predictive Consistency (ACPC) is the distance between the two weighted predicted rollouts:

$$\text{ACPC}_H(h, \tilde{h}, \mathbf{a}) = \|\bar{G}_\mathbf{a}(E_\theta(h)) - \bar{G}_\mathbf{a}(E_\theta(\tilde{h}))\|_2 = \left(\sum_{k=1}^H \alpha_k \|\Pi(\hat{z}_k) - \Pi(\hat{\tilde{z}}_k)\|_2^2 \right)^{1/2}. \quad (3)$$

In plain terms, ACPC asks how differently the model predicts the future when only the visual appearance of the observed history changes. Encoder shift $\|z - \tilde{z}\|_2$ measures the upstream displacement before prediction; ACPC measures what remains after both histories are propagated under the same actions. Sharing actions isolates visual sensitivity under a fixed intervention. It does not by itself establish correct responses to other actions.

3.2 Common-future error drift

The rollout radius has a direct consequence for prediction error. Let $Y_\mathbf{a}^H$ be one actually observed future under the same recorded action sequence, projected, stacked, and weighted exactly as in Equation (2). This target is not produced by either prediction branch and is used only for evaluation. Comparing both branches with this one future isolates the error change caused by the visual perturbation.

Proposition 1 (Common-future error-drift bound). *Define*

$$e_h = \|\bar{G}_{\mathbf{a}}(E_{\theta}(h)) - Y_{\mathbf{a}}^H\|_2, \quad e_{\tilde{h}} = \|\bar{G}_{\mathbf{a}}(E_{\theta}(\tilde{h})) - Y_{\mathbf{a}}^H\|_2.$$

Then, for every paired sample,

$$|e_{\tilde{h}} - e_h| \leq \text{ACPC}_H(h, \tilde{h}, \mathbf{a}). \quad (4)$$

The adverse increase $(e_{\tilde{h}} - e_h)_+$ obeys the same bound.

Proof. Apply the reverse triangle inequality in the weighted projected-rollout space. Both errors must use the same $Y_{\mathbf{a}}^H$. $\square$

This proposition controls a change in error, not prediction accuracy. A small ACPC value can coexist with two inaccurate predictions; it says only that the two errors to the common future cannot differ by more than the paired-rollout radius. ACPC itself does not require the realized future. The future-drift experiment uses that future only to evaluate whether ACPC is informative about the bounded quantity.

3.3 Candidate-cost drift and planner stability

The planner link is a radius-margin argument. A visual perturbation can change a fixed-pool decision only when candidate costs move far enough to cross a clean candidate gap. For candidate action sequence $\mathbf{a}^j$, let

$$x_j = \Pi(F_{\theta}^H(E_{\theta}(h), \mathbf{a}^j)), \quad \tilde{x}_j = \Pi(F_{\theta}^H(E_{\theta}(\tilde{h}), \mathbf{a}^j))$$

be the final clean and perturbed predicted representations. Their displacement $r_j = \|x_j - \tilde{x}_j\|_2$ is the final-step component of the candidate-specific $\text{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)$. If $\alpha_H > 0$, then $r_j \leq \text{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)/\sqrt{\alpha_H}$; the planner audit records r_j directly.

Proposition 2 (Squared goal-cost drift and fixed-pool stability). *Suppose the planner scores candidate j against fixed goal embedding g by $C_j = \|x_j - g\|_2^2$ and $\tilde{C}_j = \|\tilde{x}_j - g\|_2^2$. Define*

$$b_j = r_j(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2). \quad (5)$$

Then every candidate satisfies $|\tilde{C}_j - C_j| \leq b_j$.

Let w and $\tilde{w}$ be the clean and perturbed winners of the same pool. The clean-history decision regret obeys

$$0 \leq C_{\tilde{w}} - C_w \leq b_{\tilde{w}} + b_w. \quad (6)$$

If the clean winner is unique and

$$\min_{j \neq w} (C_j - C_w - b_j - b_w) > 0, \quad (7)$$

then the perturbed branch has the same winner. If $\mathcal{E}$ is the clean top- k elite set and

$$\min_{i \in \mathcal{E}, j \notin \mathcal{E}} (C_j - C_i - b_j - b_i) > 0, \quad (8)$$

then the perturbed branch has exactly the same elite set.

The quantity b_j is a candidate-specific upper bound on the movement of the squared goal cost. The regret inequality bounds the extra clean-history model cost of choosing the perturbed winner. The last two conditions subtract the allowed movements from each clean candidate gap; a positive remainder means that the gap cannot be crossed. These statements use the evaluated endpoints, not a global Lipschitz approximation. Proofs and equality cases are in Appendix A. Computing them requires both matched candidate evaluations, so they support an audit rather than a cheap pre-screen.

CEM repeatedly fits its next proposal to the current elite set. With common random numbers, identical proposals generate the same ordered candidate pool, and an identical certified elite set produces the same next proposal.

Corollary 1 (Conditional CEM stability). *Consider clean and perturbed CEM runs initialized by the same proposal and sampled with common random numbers. While their ordered candidate pools remain aligned, if Equation (8) holds at the current iteration, both runs fit the same next proposal. If it holds at every iteration, their pools, proposals, and final actions remain aligned by induction.*

The guarantee stops at the first iteration for which the elite condition fails. It concerns candidate selection and model cost under paired planner evaluations; it is not a statement about simulator return. Appendix G specifies the fixed-pool and adaptive-CEM evaluation contract.

3.4 Why low radius needs a task-proxy margin

The two propositions explain why a small same-state radius is useful, but not why it is sufficient. A constant representation makes every ACPC value zero and destroys all state distinctions. We therefore pair the radius with an explicit separation test on declared state-coordinate proxies.

First convert pairwise ACPC to a checkpoint-level radius. Throughout this section, Q_q denotes the empirical q -quantile, n is the number of logged anchors, and M is the number of perturbation draws per anchor. For anchor i , let $u_{i,0}^{\text{obs}}$ be the embedding of the last clean history observation in the projected space selected by Π , and let $u_{i,1:H}^{\text{obs}}$ be the embeddings of the next H actually observed clean frames in that same space. Define the clean-motion scale

$$s_i = Q_{0.50} \left(\left\{ \|u_{i,k}^{\text{obs}} - u_{i,k-1}^{\text{obs}}\|_2 \right\}_{k=1}^H \right). \quad (9)$$

This anchor-specific yardstick expresses rollout distances in units of ordinary clean motion; it is not a task outcome. With numerical stabilizer $\varepsilon_{\text{norm}} = 10^{-8}$ and perturbation draw m , define

$$R_i^{(m)} = \frac{\text{ACPC}_H(h_i, \tilde{h}_i^{(m)}, \mathbf{a}_i)}{s_i + \varepsilon_{\text{norm}}}, \quad \bar{R}_i = \frac{1}{M} \sum_{m=1}^M R_i^{(m)}. \quad (10)$$

The raw *Action-Conditioned Trajectory Radius* (ATR) is

$$\text{ATR}_q^{\text{raw}}(\theta) = Q_q(\{\bar{R}_i\}_{i=1}^n). \quad (11)$$

ATR is one number per checkpoint: it reports the upper-quantile radius of the same-state rollout tube. We use q90 because a mean can hide a small set of high-radius anchors. This is a descriptive summary, not a candidate-distribution tail guarantee.

For the complementary test, let y_i be the protocol-standardized logged task-state vector at the end of anchor i 's observed H -step future, and let $\ell(y_i)$ be a declared task-specific coordinate label. Let $d_{0.35}$ be the q35 of all off-diagonal Euclidean distances among the standardized endpoint states. Each eligible anchor chooses the nearest $j(i)$ with a different label and distance at most $d_{0.35}$. The predictor rolls both clean histories under the anchor actions $\mathbf{a}_i$; this reuses the anchor's recorded actions and is counterfactual only for the neighbor history. Their normalized proxy-different distance is

$$D_i^{\text{diff}} = \frac{\|\bar{G}_{\mathbf{a}_i}(E_\theta(h_i)) - \bar{G}_{\mathbf{a}_i}(E_\theta(h_{j(i)}))\|_2}{s_i + \varepsilon_{\text{norm}}}. \quad (12)$$

Using the same rollout metric, anchor action sequence, and anchor scale on both sides makes the comparison direct. The *Selective Margin Pass Rate* (SMPR) is

$$\text{SMPR}_{q,\delta}(\theta) = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbf{1}[D_i^{\text{diff}} > \text{ATR}_q^{\text{raw}}(\theta) + \delta], \quad (13)$$

where $\mathcal{I}$ contains anchors with an eligible label-crossing neighbor. The reported protocol fixes $q = 0.90$ and normalized margin $\delta = 0.10$. SMPR is the fraction of tested proxy-different pairs that remain outside the same-state tube by more than that margin.

ATR and SMPR must be read together. A constant representation makes both ATR and every D_i^{diff} zero, so it fails the SMPR test. Conversely, a model can separate the tested proxy classes while remaining visually sensitive, which yields a large ATR. These proxies test only the declared coordinate distinctions; they do not prove semantic validity, action relevance, or correct behavior. Appendix B gives the exact coordinate rules and eligible-pair counts.

3.5 Checkpoint-level calibration

The raw ATR in Equation (11) defines the tube used by SMPR. To compare augmentation levels across tasks, the experiments also use a task-relative ATR. Let θ_0 be the corresponding checkpoint trained without visual augmentation for the same task, training run, and model family. Its raw ATR is positive in the evaluated data, so

$$\widetilde{\text{ATR}}_{q,v}(\theta; \theta_0) = \frac{\text{ATR}_{q,v}^{\text{raw}}(\theta)}{\text{ATR}_{q,v}^{\text{raw}}(\theta_0)}. \quad (14)$$

The raw quantity remains inside Equation (13); the relative quantity equals one at the reference checkpoint and is used for cross-checkpoint calibration.

For model family $\mathcal{F}$ and visual shift v , thresholds t_R and t_M combine the two sides. With $\varepsilon_{\text{score}} = 10^{-12}$, define

$$S_{\mathcal{F},v}(\theta; \theta_0) = \min \left\{ \frac{t_R - \widetilde{\text{ATR}}_{q,v}(\theta; \theta_0)}{|t_R| + \varepsilon_{\text{score}}}, \frac{\text{SMPR}_{q,\delta,v}(\theta) - t_M}{|t_M| + \varepsilon_{\text{score}}} \right\}. \quad (15)$$

Thus $S \geq 0$ exactly when both weak inequalities pass: relative ATR is at most t_R and SMPR is at least t_M . Computing the score for a fixed checkpoint uses no task outcome. Planning success rates enter only when the thresholds are selected on source tasks and evaluated on other tasks. Thresholds are selected separately within each model family; the equations do not imply shared raw thresholds across architectures.

For a comparison with reference checkpoint θ_0 , define

$$\Delta S_{\mathcal{F},v}(\theta_1; \theta_0) = S_{\mathcal{F},v}(\theta_1; \theta_0) - S_{\mathcal{F},v}(\theta_0; \theta_0). \quad (16)$$

A positive ΔS means only that θ_1 improves on its reference under the same calibrated scoring map. It does not assign an absolute robustness label to either checkpoint.

Table 1 summarizes the chain. The future-drift experiment evaluates whether ACPC is informative about the quantity bounded by Proposition 1; the CEM experiment analyzes the candidate-cost and selection quantities in Proposition 2 and corollary 1; and the cross-task, PLDM, and cross-stressor analyses evaluate the calibrated ATR–SMPR score. These are related but distinct empirical claims.

Table 1: Reader guide to the ACPC diagnostic chain.

Object	Plain-language meaning	Empirical role
ACPC	Distance between same-state clean and perturbed predicted futures under shared actions.	Future-error and planner analyses
b_j	Candidate-specific upper bound on squared-cost movement.	Fixed-pool planner audit
ATR^{raw}	q90 radius of the normalized same-state rollout tube.	Tube used by SMPR
D_i^{diff}	Rollout distance to a local state with a different state-coordinate proxy label.	Proxy-separation test
SMPR	Fraction of proxy-different pairs beyond the raw tube plus margin.	Constant-collapse guard
$\widetilde{\text{ATR}}, S$	Task-relative radius and the worse calibrated radius/margin condition.	Sweep and cross-task screening
ΔS	Change in calibrated selective score relative to a reference checkpoint.	Cross-stressor ordering

4 Experiments

Figure 1 provides an overview of planning success and the two checkpoint-level diagnostics across the Gaussian sweep. We first state the common evaluation protocol, then examine prediction, planning, and transfer claims separately.

4.1 Evaluation setup

Models, tasks, and planning evaluation. LeWM [19] is the primary model family; PLDM [25, 26] provides a second joint-embedding dynamics architecture. We evaluate TwoRoom navigation, PushT planar manipulation, Reacher arm control, and OGBench-Cube (Cube) 3D manipulation. We follow the standard LeWM latent-space MPC evaluation: CEM optimizes candidate action sequences with the frozen world model, and performance is the percentage of evaluation episodes that reach the task goal, reported as *planning success rate*. The main evaluation adds Gaussian noise $\sigma = 0.08$ to observations while keeping the goal image clean. Additional evaluations use Gaussian blur ($k = 15$) and resize (scale 0.25).

For each model family, the Gaussian-augmentation sweep contains a checkpoint trained without noise augmentation and checkpoints trained with full-sequence noise $\sigma_{\text{max}} \in \{0.01, \dots, 0.08\}$. Each LeWM task–augmentation setting is trained with seeds 3072/3073/3074 and evaluated with seeds 42/43/44 using 100 episodes per evaluation seed. Evaluation seeds are conditional measurement replicates; the independently trained model is the replication unit. The PLDM sweep contains one checkpoint for each of the 36 task–augmentation settings. These checkpoints use the same evaluation seeds and episode count.

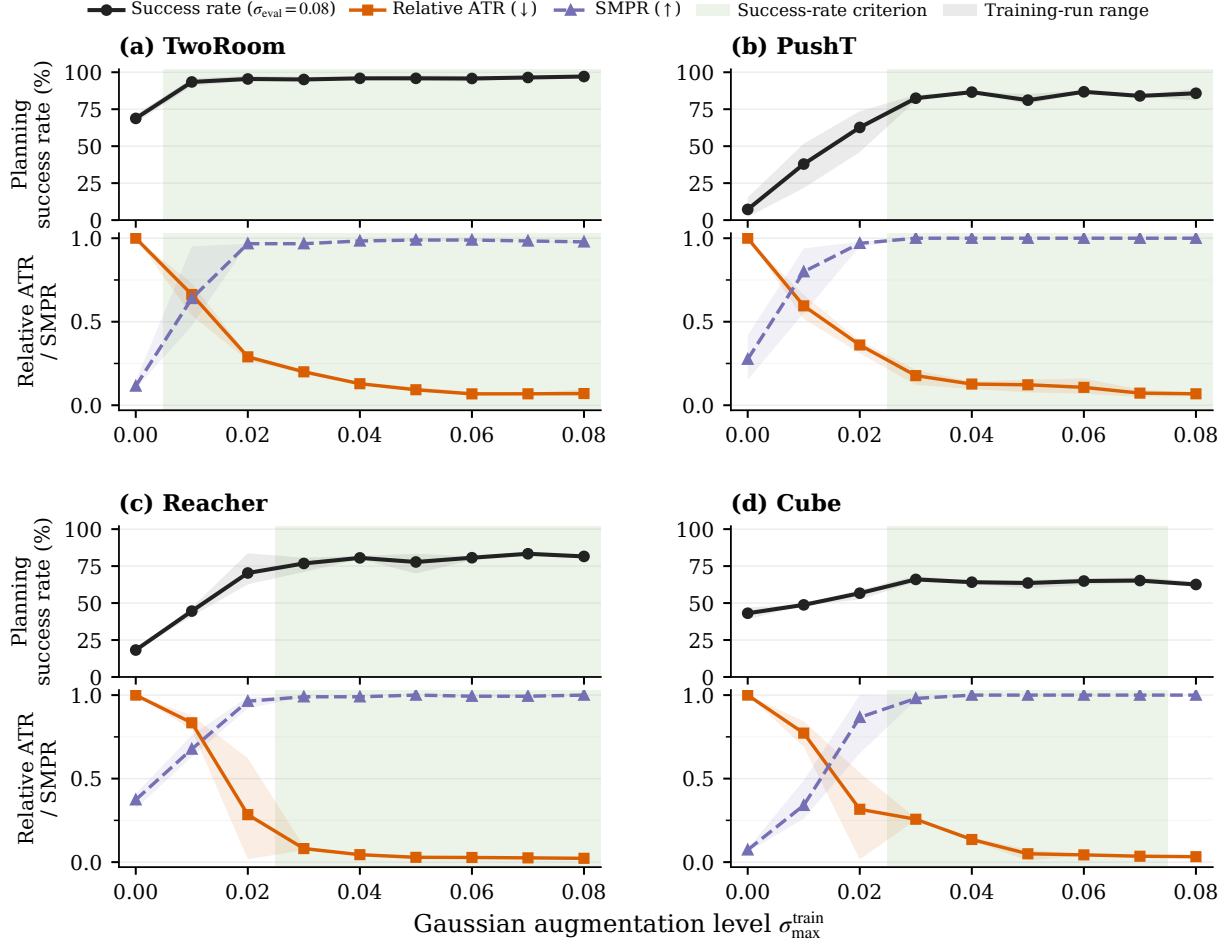


Figure 1: Planning success rate and diagnostic measurements across the Gaussian-augmentation sweep. Success rates use evaluation noise $\sigma = 0.08$; ATR is the q90 same-state predictive radius relative to the checkpoint trained without augmentation (value 1). SMPR is the fraction of local state-coordinate proxy pairs whose different-state distance exceeds the raw q90 tube by 0.10. Lines show across-run means, shaded ranges span the run means, and green bands mark levels that satisfy the success-rate criterion in a majority of runs. Lower ATR and higher SMPR are favorable.

Diagnostic measurements. ATR uses 100 logged anchors, five perturbation draws, eight predicted steps, uniform horizon weights, within-anchor draw averaging, and a q90 summary across anchors. SMPR uses the same-state q90 tube, a global q35 radius among standardized endpoint states for selecting coordinate-proxy pairs, and margin 0.10. Appendix B gives the complete normalization and threshold protocol. The raw ATR defines the tube in SMPR; checkpoint thresholding uses the task-relative ATR in Equation (14).

Prediction and CEM analyses. To test prediction-error changes, we use 16 trajectory groups per task, four perturbation severities, and two perturbation draws. The target is the absolute change in eight-step latent prediction error between nominal and perturbed histories. A ridge model is fitted on 15 groups and evaluated on the remaining group, rotating through all 16. Every regression uses perturbation severity, encoder shift, action magnitude, and nominal dynamics. The regressions differ only in the added ACPC feature: one-step ACPC; the strongest of three eight-step controls using zeroed actions, actions from another trajectory, or time-shuffled actions; or eight-step ACPC using the recorded actions.

For the CEM analysis, checkpoints trained without noise augmentation and with $\sigma_{\max} = 0.08$ are evaluated on the same candidate pools. The reduced-budget analysis uses 100 histories per task and training run, $K = 64$, top-eight elites, and eight CEM steps. The full-budget analysis uses the same 16 histories per task with the deployed $K = 300$, top-30, 30-step configuration. Within each training run, ridge models are fitted on three tasks and evaluated on the fourth. We test whether candidate-conditioned five-step ACPC improves prediction

of the maximum candidate-cost change, first-action RMS, and clean-history decision regret beyond perturbation severity, training condition, one-step ACPC, and the nominal top-1 margin.

Threshold evaluation. A Gaussian-sweep checkpoint meets the success-rate criterion when it attains at least 80% of the improvement from the checkpoint trained without augmentation to the best checkpoint at evaluation noise $\sigma = 0.08$, while losing no more than five percentage points on clean observations. We select (t_R, t_M) on every nonempty proper subset of the four tasks and apply the thresholds unchanged to the remaining tasks. This gives 14 directional source/evaluation partitions. Metrics first average training runs within each evaluation task and then weight tasks equally; checkpoint rows are not treated as independent replicates. Success rates are used to select and evaluate thresholds, but they are not inputs to ACPC, ATR, or SMPR.

For blur and resize, each task uses the Gaussian thresholds selected on the other three tasks. We compare all 24 pairs of checkpoints trained without augmentation and with $\sigma_{\max} = 0.08$. A pair is positive when success rate under blur or resize improves by at least five percentage points and clean success rate decreases by no more than five points. No blur- or resize-specific adjustment is made, and the transfer analysis does not rank encoder shift or one-step ACPC.

4.2 Planning performance under observation noise

At evaluation noise $\sigma = 0.08$, LeWM checkpoints trained without noise augmentation lose 25.89 ± 2.38 , 74.56 ± 5.55 , 41.00 ± 1.44 , and 22.11 ± 2.78 percentage points in success rate on TwoRoom, PushT, Reacher, and Cube, respectively. Gaussian augmentation improves performance over a range of training levels whose location differs by task. Figure 1 compares success rate with ATR and SMPR across the complete sweep.

Every augmentation level that meets the success-rate criterion has lower ATR and higher SMPR than the corresponding checkpoint trained without augmentation. This comparison does not imply that either diagnostic changes monotonically with augmentation strength. It motivates the combined diagnostic, but does not by itself show that ACPC uses action information or tracks a planning-relevant quantity. The next two experiments examine those questions.

4.3 Predicting error changes under visual perturbations

We ask whether ACPC helps predict how much a visual perturbation changes an eight-step prediction error, the quantity bounded by Proposition 1. Both predictions are compared with the same observed future, and the target is the absolute difference between their errors. Every regression uses the same perturbation severity, encoder shift, action magnitude, and nominal-dynamics features. The regressions differ only in their ACPC feature: one-step ACPC; the strongest of three same-horizon destroyed-action controls using zeroed actions, actions from another trajectory, or time-shuffled actions; or eight-step ACPC under the recorded actions. The same-horizon comparison isolates the recorded action information from rollout length.

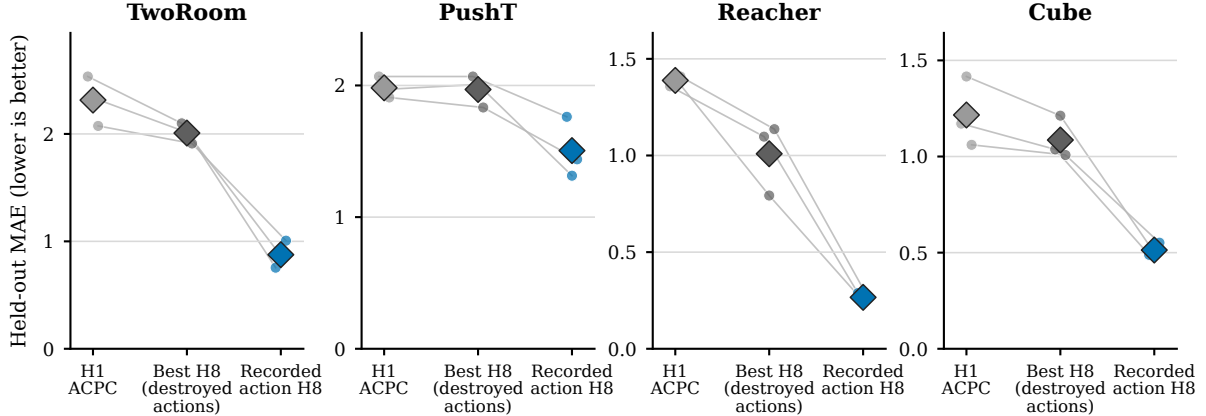


Figure 2: All regressions predict the same target: the absolute change in prediction error against one observed eight-step future. The vertical axis is held-out MAE, so lower is better. Each task panel reports regressions using one-step ACPC, the best same-horizon destroyed-action control, and recorded-action eight-step ACPC. Small points are individual training runs, thin lines join results from the same run, and diamonds are task means. All regressions use identical base covariates. The control is selected from zeroed, cross-trajectory, and time-shuffled actions. Latent-error scales are task-specific, so comparisons should be made within a panel.

The regression using recorded-action eight-step ACPC has the lowest held-out MAE in all 12 task-run cells. Relative to one-step ACPC, the equal-task MAE reduction is $55.9 \pm 4.7\%$; relative to the strongest same-horizon control it is $51.3 \pm 3.5\%$ (mean $\pm$ sample standard deviation across run-level equal-task summaries). These results concern the ability to predict a change in eight-step error; they do not compare the absolute accuracy of one-step and eight-step forecasts. Appendix C reports the task-level values and selected controls.

4.4 ACPC and CEM decisions

We next test whether ACPC is informative about the CEM quantities in Proposition 2. Across 9,600 reduced-budget records, the candidate-cost bound has no violations, and the winner and elite-set certificates never assert stability when the corresponding set changes. Identity probes give zero five-step ACPC and zero first-action RMS. Whenever the elite-set certificate remains valid, the nominal and perturbed CEM updates also remain aligned, as required by Corollary 1.

We then ask whether candidate-conditioned five-step ACPC adds predictive information beyond perturbation severity, training condition, one-step ACPC, and the nominal top-1 margin. Table 2 reports the reduction in held-out prediction error, and Fig. 3 shows paired held-out errors for the baseline and expanded regressions.

Table 2: Held-out prediction gain from candidate-conditioned five-step ACPC in the reduced-budget CEM audit (9,600 history records). Both ridge models include perturbation severity, training condition, one-step ACPC, and the nominal top-1 margin; the expanded model adds five-step ACPC. Within each independent training run, held-out MAE on the fitted $\log(1 + \text{target})$ response is averaged equally across the four leave-one-task-out evaluations for each model, and the relative decrease between those two equal-task MAEs is then computed. Entries report the across-run mean $\pm$ sample standard deviation of these run-level decreases; the final column counts positive task-run cases out of 12. Higher is better. Decision regret is measured in the model’s squared latent goal cost, not simulator return.

Prediction target	Held-out MAE decrease (%, mean $\pm$ sample SD, $\uparrow$)	Positive task-run cases (/12)
Maximum fixed-pool cost movement	7.9 ± 1.4	8/12
Adaptive-CEM decision regret	15.2 ± 2.0	12/12
Adaptive-CEM first-action change (RMS)	1.1 ± 0.5	10/12

Adding candidate-conditioned five-step ACPC reduces held-out MAE for maximum fixed-pool cost movement by $7.9 \pm 1.4\%$; the reduction is positive on three of the four tasks. For adaptive CEM, it reduces held-out MAE for positive decision regret by $15.2 \pm 2.0\%$, with an improvement in every task-run cell. The $1.1 \pm 0.5\%$ reduction for first-action change is too small to support an incremental claim. In all three analyses, the rank association is higher for five-step than for one-step ACPC in every task-run cell.

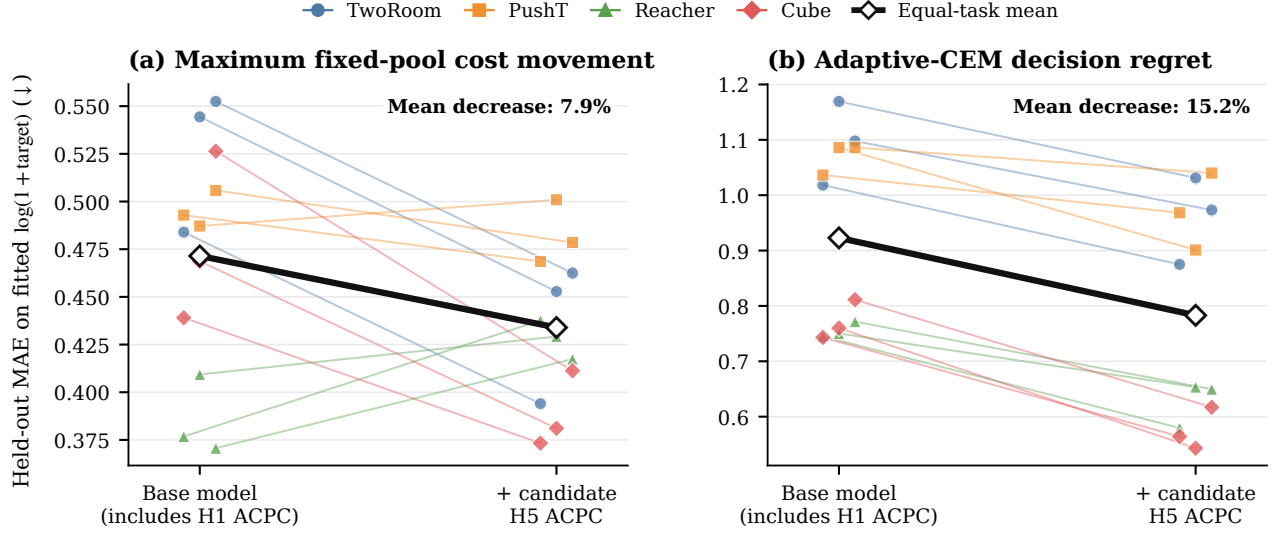


Figure 3: Direct held-out prediction errors for two planner-facing quantities; lower is better. Each panel compares two nested ridge models fitted and evaluated on the same task–run split, without and with candidate-conditioned five-step ACPC. The base model already includes perturbation severity, training condition, one-step ACPC, and the nominal top-1 margin. Thin lines connect the two errors for the same task–run evaluation; the heavy black pair is the across-run mean of the run-level equal-task MAEs. The panel annotation is the across-run mean of the run-level relative decreases reported in Table 2. Errors are measured on the fitted $\log(1 + \text{target})$ scale, and axis ranges are response-specific. Decision regret is extra squared latent goal cost under the frozen model, not simulator return.

At perturbation severity 0.08, the task-level mean rate of retaining the best fixed-pool candidate is 0.92–0.96 after training with Gaussian augmentation, compared with 0.02–0.13 without augmentation. Mean positive decision regret is also lower with augmentation in every task–run cell: 3.64 ± 0.68 versus 227.04 ± 16.92 at reduced budget and 2.65 ± 1.43 versus 244.89 ± 27.08 at full budget (equal-task mean $\pm$ sample SD across runs). Because latent cost scales differ by task, these contrasts are reported descriptively rather than pooled as one effect. Appendix G gives the task-level values.

4.5 Cross-task threshold transfer

We ask whether ATR and SMPR thresholds selected on some tasks can screen checkpoints from tasks not used for calibration. We select thresholds on every nonempty proper subset of the four tasks and apply them unchanged to all remaining tasks. This gives 14 directional source/evaluation partitions (Fig. 4).

Balanced accuracy is 0.855, 0.900, and 0.900 when thresholds are selected from one, two, and three source tasks, respectively. Corresponding precision/recall values are 0.910/0.854, 0.913/0.953, and 0.913/0.953. Mean absolute onset errors are 1.1, 0.5, and 0.5 grid steps; the largest individual partition–task–run errors are 5, 1, and 1 steps. With one source task, balanced accuracy ranges from 0.723 to 0.913 across the four possible sources, showing that task composition matters. Two and three source tasks perform similarly on average in this four-task study. Thus the evaluation supports transfer of the threshold-selection procedure across these tasks, but not one numerical threshold for every task or model family. Appendix D reports every partition.

Cross-task transfer of calibrated thresholds

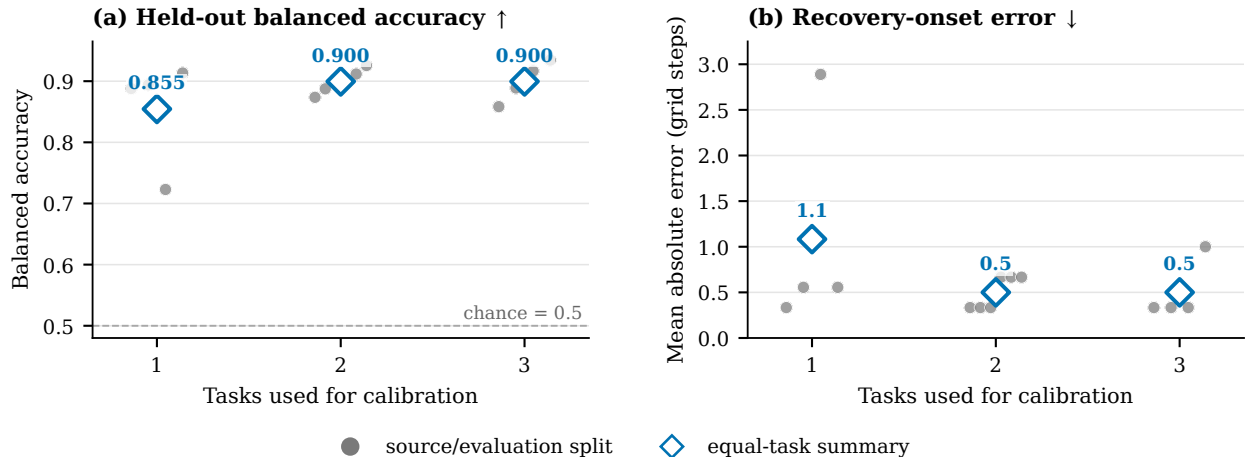


Figure 4: Cross-task transfer of ATR-SMPR thresholds over all 14 source/evaluation partitions. Thresholds are selected using source tasks only and then applied unchanged to the remaining tasks; a checkpoint passes when relative ATR $\leq t_R$ and SMPR $\geq t_M$. Panel (a) reports balanced accuracy on held-out checkpoints (higher is better; chance is 0.5). Panel (b) reports the absolute gap between the first augmentation level accepted by the diagnostic and the first level meeting the success-rate criterion, measured in Gaussian-augmentation grid steps (lower is better; one step is 0.01 in training-noise standard deviation). Gray points are individual source/evaluation partitions, and diamonds are equal-task summaries for each number of calibration tasks.

4.6 Application to PLDM

We apply the same paired-history construction, eight-step rollout, ATR normalization, SMPR guard, success-rate criterion, and threshold-selection procedure to the complete PLDM Gaussian sweep. The analysis uses PLDM’s encoder and predictor but does not depend on LeWM layers or training losses. For each PLDM evaluation task, thresholds are selected on the other three PLDM tasks and then fixed for its nine checkpoints.

Pooling the four evaluation branches gives 0.836 balanced accuracy, 0.789 precision, and 0.882 recall. Task balanced accuracies are 0.938, 0.750, 0.500, and 0.875 for TwoRoom, PushT, Reacher, and Cube, respectively; Reacher is therefore the unresolved boundary case. After normalizing ATR within each PLDM task, applying the corresponding LeWM source-task thresholds gives the same aggregate values: 0.836 balanced accuracy, 0.789 precision, and 0.882 recall. This numerical agreement on the present grid does not imply that thresholds are architecture-invariant. The shared equations and evaluation procedure apply to both LeWM and PLDM, while calibration remains model-family specific.

Table 3: Applying the same ACPC analysis to the complete PLDM sweep of four tasks and nine augmentation levels. The first row selects thresholds on the other three PLDM tasks. The second uses the corresponding LeWM thresholds after normalizing ATR within each PLDM task. Metrics include all 36 PLDM evaluation rows; raw thresholds are not assumed to match across model families.

Calibration	BA	Precision	Recall	False pass	False miss
PLDM thresholds, other three tasks	0.836	0.789	0.882	4	2
LeWM thresholds, PLDM-normalized	0.836	0.789	0.882	4	2

4.7 Transfer to blur and resize

Finally, we ask whether a Gaussian-calibrated change in the selective score orders checkpoint pairs in the same direction as planning success under new visual shifts. For each task, we apply thresholds selected on the other three Gaussian-noise tasks to checkpoint pairs evaluated under blur and resize. No blur or resize result is used to select or adjust a threshold.

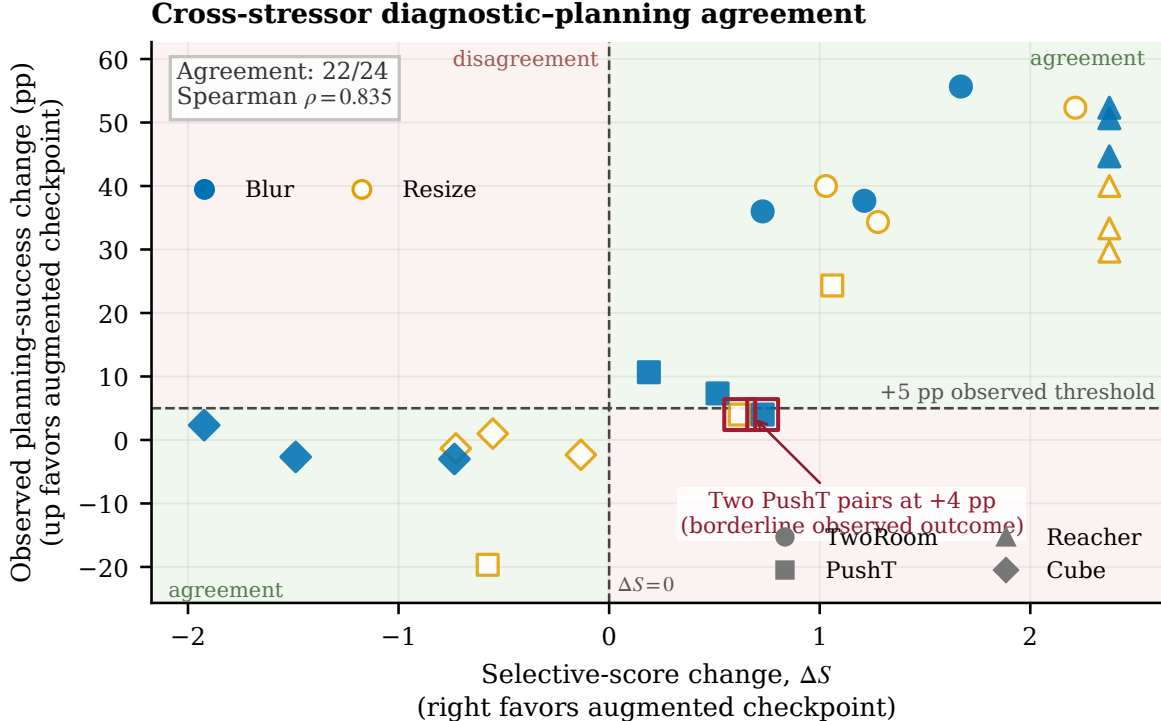


Figure 5: Agreement between the Gaussian-calibrated selective score and observed planning changes under blur and resize. Each of the 24 points compares a checkpoint trained with Gaussian augmentation ($\sigma_{\max} = 0.08$) with the corresponding checkpoint trained without augmentation. A point to the right means the selective score favors the augmented checkpoint; a higher point means its planning success rate is higher. The diagnostic predicts improvement when $\Delta S > 0$; the observed positive criterion requires at least a five-percentage-point success gain and no more than a five-point clean loss. Background regions show agreement or disagreement between these decisions. Marker shape denotes task and color/fill denotes the visual shift; blur and resize are each evaluated at one fixed severity.

The sign of ΔS agrees with the observed label on 22 of 24 pairs and reaches 0.889 balanced accuracy, 0.882 precision, and 1.000 recall. The continuous score change has Spearman’s $\rho = 0.835$ with the change in success rate. Balanced accuracy is 0.875 for blur and 0.900 for resize. The two discordant cases are both PushT changes of exactly four percentage points, which fall just below the fixed five-point positive label while the selective score improves. These results support relative checkpoint ordering for these two visual shifts at the tested severity. They do not establish an absolute robustness classifier.

5 Discussion and limitations

The prediction experiment asks whether ACPC is informative about the error change bounded in Proposition 1. Recorded-action eight-step ACPC adds predictive information beyond common covariates that already include encoder shift, and it predicts the change in latent error better than one-step ACPC and the three same-horizon destroyed-action controls. Because every regression uses the same eight-step target, the result cannot be attributed only to comparing different rollout horizons. The direction is consistent across all tasks and runs, although the available runs do not support a precise estimate of population-level training variability.

The CEM experiment verifies the deterministic cost bound and tests whether candidate-conditioned ACPC explains decision changes beyond simpler features. Its incremental gain is consistent for decision regret, positive on three of four tasks for maximum cost change, and negligible for first-action RMS. These are model-cost and candidate-selection results, not environment success rates. The deployed-budget analysis also uses only 16 shared histories per task, and the evaluation does not include matched runs without diagnostic computation; it therefore does not estimate incremental wall-clock overhead.

Cross-task evaluation covers every source/remaining-task partition. The plateau from two to three source tasks, together with the wide one-source range, shows that calibration depends on task composition. The PLDM

experiment extends the equations and analysis procedure to a second joint-embedding dynamics architecture while leaving numerical calibration model-family specific; the present grid does not establish architecture-invariant thresholds. Blur and resize test a narrower form of transfer: the Gaussian-calibrated score preserves useful relative checkpoint ordering at one severity for each visual shift.

Across the experiments, SMPR remains a guard against constant-representation failure on the tested proxies, the cost certificates require candidate-wise paired evaluation, and the adaptive result is conditional on pool alignment until an elite certificate fails. Stable but incorrect predictions remain possible. Important next steps are direct task outcomes for ACPC-informed planner interventions, multiple severities for blur and resize, richer different-state proxies, and additional model families with different objectives and planner interfaces.

6 Conclusion

ACPC measures how a same-state visual perturbation changes a world model’s predicted trajectory under fixed actions. Exact diagnostic inequalities relate this change to prediction error and squared goal cost, and a state-coordinate margin rejects constant-representation collapse on the tested proxies. Across four tasks, recorded-action ACPC adds information beyond one-step and same-horizon action controls, relates to CEM cost and decision changes, and supports thresholds that transfer to held-out tasks. The same analysis applies to PLDM, and a Gaussian-calibrated score retains useful checkpoint ordering under blur and resize. Numerical thresholds remain model-family specific.

References

- [1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.
- [2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohov, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [3] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.
- [4] Jiaheng Chen. ATM: Action-consistency transfer matrix for diagnosing and improving latent world models, 2026.
- [5] Yuxin Chen, Jianglan Wei, Chenfeng Xu, Boyi Li, Masayoshi Tomizuka, Andrea Bajcsy, and Ran Tian. Reimagination with test-time observation interventions: Distractor-robust world model predictions for visual model predictive control, 2025.
- [6] Tom Dupuis, Jaonary Rabarisoa, Quoc-Cuong Pham, and David Filliat. VIBR: Learning view-invariant value functions for robust visual control. In *Proceedings of The 2nd Conference on Lifelong Learning Agents*, volume 232 of *Proceedings of Machine Learning Research*, pages 658–682. PMLR, 2023.
- [7] T. W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.
- [8] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 2019.

- [9] Hafez Ghaemi, Eilif Benjamin Muller, and Shahab Bakhtiari. seq-JEPA: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [10] Christopher Grimm, Andre Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.
- [11] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.
- [12] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [13] Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 13611–13617, 2021.
- [14] David Klindt, Yann LeCun, and Randall Balestriero. When does LeJEPA learn a world model?, 2026.
- [15] Ilya Kostrikov, Denis Yarats, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [16] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.
- [17] Etai Littwin, Omid Saremi, Madhu Advani, Vimal Thilak, Preetum Nakkiran, Chen Huang, and Joshua Susskind. How JEPA avoids noisy features: The implicit bias of deep linear self distillation networks, 2024.
- [18] Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation, 2026.
- [19] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- [20] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.
- [21] Bo-Kai Ruan, Teng-Fang Hsiao, Ling Lo, and Hong-Han Shuai. Is the future compatible? diagnosing dynamic consistency in world action models, 2026.
- [22] Finn Rasmus Schäfer, Korbinian Moller, Yuan Gao, Christian Oefinger, Sebastian Schmidt, and Johannes Betz. Imagined rollouts are kinematic, not dynamic: A diagnosis of long-horizon world-model failure, 2026. RSS Robot World Model Workshop 2026.
- [23] Gawon Seo, Dongwon Kim, and Suha Kwak. ACID: Action consistency via inverse dynamics for planning with world models, 2026.
- [24] Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [25] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022.
- [26] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Stress-testing offline reward-free reinforcement learning: A case for planning with latent dynamics models. In *WRL@ICLR 2025*, 2025.

- [27] Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026.
- [28] Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint embedding vs reconstruction: Provable benefits of latent space prediction for self-supervised learning, 2025.
- [29] Claas A. Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-Massoud Farahmand. Calibrated value-aware model learning with probabilistic environment models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 61745–61768. PMLR, 2025.
- [30] Zijie Wang, Wei Zhang, Weiming Zhang, Fanqi Zhang, Xiao Tan, Yipeng Qin, and Guanbin Li. World models as group actions, 2026.
- [31] Han Yan, Zishang Xiang, Zeyu Zhang, and Hao Tang. MWM: Mobile world models for action-conditioned consistent prediction, 2026.
- [32] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
- [33] Amy Zhang, Rowan T. McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [34] Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong, Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, Chenxi Bao, Tao Yu, Jingjing Zhou, and Jungang Xu. Delta-JEPA: Learning action-sensitive world models via latent difference decoding, 2026.

A Candidate-Cost and Fixed-Pool Analysis

Proof of Proposition 2. For one candidate, suppress the index and factor the squared-distance difference:

$$|\tilde{C} - C| = \left| \|\tilde{x} - g\|_2^2 - \|x - g\|_2^2 \right| \quad (17)$$

$$= \left| \langle \tilde{x} - x, \tilde{x} + x - 2g \rangle \right| \quad (18)$$

$$\leq \|\tilde{x} - x\|_2 \|\tilde{x} + x - 2g\|_2 \quad (19)$$

$$\leq r(\|x - g\|_2 + \|\tilde{x} - g\|_2) = b. \quad (20)$$

The first inequality is Cauchy–Schwarz and the second is the triangle inequality. Thus, for nominal winner w and competitor j ,

$$\tilde{C}_j - \tilde{C}_w \geq C_j - C_w - |\tilde{C}_j - C_j| - |\tilde{C}_w - C_w| \geq C_j - C_w - b_j - b_w.$$

For the perturbed winner $\tilde{w}$,

$$C_{\tilde{w}} \leq \tilde{C}_{\tilde{w}} + b_{\tilde{w}} \leq \tilde{C}_w + b_{\tilde{w}} \leq C_w + b_w + b_{\tilde{w}},$$

which proves Equation (6); nonnegativity follows from the optimality of w on the nominal pool. Positivity of Equation (7) makes every perturbed competitor strictly more costly than w . The same argument for every $i \in \mathcal{E}$ and $j \notin \mathcal{E}$ proves that Equation (8) preserves the exact elite set. Strict inequalities avoid relying on implementation-specific tie breaking.

Proof of Corollary 1. At iteration zero, the proposals are identical by assumption. Common random numbers therefore produce the same ordered action pool. If the elite certificate holds, both branches select the same action vectors as elites. The CEM mean/variance update is a deterministic function of those vectors, so the next proposals are identical. Repeating this argument establishes pool and proposal alignment by induction. If a certificate fails, equality of the elite sets is unknown; the induction stops and no later shared-pool statement is made.

Relation to the weighted rollout radius. The planner bound uses the displacement at the cost readout horizon. For the weighted stack in Equation (3), $\sqrt{\alpha_H} r_j \leq \text{ACPC}_H$ whenever $\alpha_H > 0$. The formal planner panel records r_j directly, avoiding the looser $1/\sqrt{\alpha_H}$ conversion. This does not require a future target, but it does require evaluating the candidate under both matched histories.

Sharpness without additional assumptions. Neither principal bound admits a uniformly smaller right-hand side from the available quantities alone. For a unit vector u , common target $Y = 0$, and two stacked predictions au and bu with $a, b \geq 0$, the reverse-triangle bound in Equation (4) holds with equality. Likewise, setting $g = 0$, $x = au$, and $\tilde{x} = bu$ makes $|\tilde{C} - C| = |b - a|(a + b) = r(\|x - g\|_2 + \|\tilde{x} - g\|_2)$.

B Evaluation and Diagnostic Protocol

These details define the fixed evaluation and diagnostic protocol used by the main tables.

LeWM is evaluated on PushT, TwoRoom, Reacher, and Cube with evaluation seeds 42/43/44 and 100 episodes per seed. The main evaluation perturbs observation pixels with Gaussian noise at standard deviation 0.08 while keeping the goal image clean. Checkpoints trained with noise use full-sequence input-side Gaussian augmentation with $\sigma_{\max} \in \{0.01, \dots, 0.08\}$.

PLDM uses the same four-task, nine-checkpoint Gaussian grid, evaluation seeds, trajectory count, paired-history construction, $H = 8$ rollout statistic, and SMPR definition. For its cross-task analysis, ATR is divided by its value for the corresponding PLDM checkpoint trained without augmentation, and thresholds are selected on the other three PLDM tasks. The evaluation task’s success rates never enter threshold selection. A second test applies the corresponding LeWM threshold pair after this within-task PLDM normalization. Raw thresholds are not assumed to be shared across model families.

Within each task and training run, let B and M be the success rates at evaluation noise $\sigma = 0.08$ for the checkpoint trained without augmentation and the best checkpoint in the sweep. A checkpoint meets the success-rate criterion when its noisy-observation success rate is at least $B + 0.8(M - B)$ and its clean success rate is no more than five percentage points below that of the checkpoint trained without augmentation. The green spans in Figure 1 require this criterion for a majority of runs.

ATR is computed for each task, training run, and checkpoint from one weighted-stacked horizon radius per anchor. The default is $H = 8$ with uniform $\alpha_k = 1/H$. Each radius is divided by that anchor’s q50 clean observed-transition scale, including the transition from the last history observation to the first future observation; multiple noise draws are first averaged within the anchor, and q90 is then taken across anchors. This is the raw ATR in Equation (11). It defines the same-state tube used by SMPR. For the sweep display and within-family cross-task calibration, raw ATR is divided by the corresponding no-augmentation value as in Equation (14) before thresholds or run summaries are computed.

SMPR follows Equation (13). For each eligible anchor, the pair is its nearest proxy-label-crossing neighbor whose standardized state distance is no greater than the global q35 radius. The two clean histories are rolled out under the anchor’s recorded action sequence, their distance is normalized by the anchor’s clean transition scale, and the pair passes only when that distance is strictly greater than the raw q90 same-state tube plus 0.10. Across-run means and dispersions are descriptive rather than population inference.

Table 4 makes the pairing rule explicit. The logged state is taken at the end of the observed eight-step future and standardized coordinate-wise by the dataset preprocessor. We compute all off-diagonal Euclidean distances in that task’s full standardized state vector and use their global q35 as the neighborhood radius. Proxy labels use median splits computed within the fixed 100-endpoint anchor batch (anchor seed 9101); each label therefore describes the endpoint reached by that trajectory’s own recorded actions. After neighbor selection, both starting histories are rolled out under the anchor’s actions for the latent-distance test. An anchor is skipped only when no label-crossing state lies inside the neighborhood. The eligible counts are constant across the evaluated checkpoints and training runs because pairing is fixed by the logged states.

Table 5: Summary of the nine-level Gaussian-augmentation sweep. “Best” denotes the augmentation level with the highest mean planning success rate at evaluation noise $\sigma = 0.08$; Fig. 1 shows every level. Relative ATR is divided by its value without noise augmentation and is lower-is-better; SMPR is reported on its original scale and is higher-is-better.

Task	No-aug. success (%)	Best success (%)	Best $\sigma_{\max}$	Relative ATR	SMPR	Levels meeting criterion
TwoRoom	68.8	97.1	0.08	1.00→0.07	0.11→0.98	0.01–0.08
PushT	7.2	86.8	0.06	1.00→0.11	0.28→1.00	0.03–0.08
Reacher	18.2	83.3	0.07	1.00→0.03	0.37→0.99	0.03–0.08
Cube	43.1	66.0	0.03	1.00→0.26	0.07→0.98	0.03–0.07

Table 6: Predicting the change in eight-step latent error under a visual perturbation. All regressions use the same target and base features; they differ only in the ACPC feature. The recorded-action eight-step feature is compared with one-step ACPC and with the strongest of three same-horizon features using zeroed actions, actions from another trajectory, or time-shuffled actions. Values are relative reductions in held-out MAE after rotating evaluation over 16 trajectory groups; higher is better.

Training run	vs. one-step ACPC	vs. same-horizon control	Task-run cells improved
seed 3072	55.5%	51.4%	4/4
seed 3073	60.8%	54.8%	4/4
seed 3074	51.4%	47.7%	4/4
mean $\pm$ SD	55.9 $\pm$ 4.7%	51.3 $\pm$ 3.5%	12/12

Table 4: Implemented state-coordinate proxies for SMPR. The state is taken at the eighth observed future step and standardized coordinate-wise with full-dataset statistics. Counts give eligible and skipped anchors in the fixed 100-anchor audit; these labels are not semantic or action-relevance annotations.

Task	HDF5 key (dim.)	Implemented proxy label $\ell(y)$	Eligible / skipped
TwoRoom	pos_agent (2)	Median split of the standardized agent x coordinate.	61 / 39
PushT	state (7)	Three median bits from standardized block x , block y , and block angle (slice state [2:5]).	98 / 2
Reacher	observation (6)	Three median bits from the two standardized joint-velocity coordinates and their Euclidean norm (slice observation [4:6]).	100 / 0
Cube	observation (28)	For standardized observation $\bar{o}$, let $r = \bar{o}_{0:3} - \bar{o}_{25:28}$; use median bits of r_1 , r_2 , and $\ r\ _2$.	100 / 0

For cross-task evaluation, candidate thresholds are $t_R \in \{0.05, 0.075, 0.10, 0.15, 0.20, 0.30\}$ for task-relative ATR and $t_M \in \{0.80, 0.85, 0.90, 0.95\}$ for SMPR. Within each source-task subset, the selection criterion lexicographically minimizes mean absolute onset error, false-early and false-late counts, then maximizes balanced accuracy. The selected pair is applied without modification to every evaluation task. No evaluation-task label or blur/resize outcome enters threshold selection.

All summary figures and tables are rebuilt deterministically by the scripts documented in `paper1/scripts/REA_DME.md`. Checkpoint-level diagnostics additionally require the released checkpoints and datasets. Machine-readable summaries record training seeds, evaluation seeds, and protocol hashes.

C Prediction-Error Scope and Controls

The common-future inequality in Proposition 1 has zero numerical violations in every logged eight-step and candidate-conditioned five-step task×run×training-condition cell. This is an implementation invariant, not an independent statistical success count.

Logged-future prediction and candidate planning are different estimands: the former predicts an independent action-matched observed future after common covariates are supplied, whereas the latter compares candidates from the actual planner pool. We do not infer one result from the other. The separate panel in Section 4.4 supplies planner-facing evidence with same-pool, candidate-conditioned one- and five-step features.

The prediction-error analysis is restricted to checkpoints trained without noise augmentation and to the listed visual probes. Every run improves on all four tasks; the across-run mean and sample standard deviation summarize repeatability rather than a precisely estimated population distribution.

Table 7: Held-out MAE for predicting the absolute change in eight-step latent prediction error. Each row contains 16 trajectory groups. The recorded-action column uses eight-step ACPC with the observed actions; the control column uses the strongest of three eight-step features with zeroed actions, actions from another trajectory, or time-shuffled actions. Lower is better.

Task	seed	one-step	same-horizon	control	recorded-action 8-step	control type	win blocks
TwoRoom	3072	2.535		2.099	0.755	time-shuffled actions	15/16
TwoRoom	3073	2.336		2.015	0.866	time-shuffled actions	15/16
TwoRoom	3074	2.075		1.911	1.006	cross-trajectory actions	13/16
PushT	3072	2.068		2.068	1.762	time-shuffled actions	11/16
PushT	3073	1.969		2.008	1.315	zero actions	13/16
PushT	3074	1.909		1.833	1.439	zero actions	13/16
Reacher	3072	1.358		1.097	0.288	time-shuffled actions	16/16
Reacher	3073	1.399		0.793	0.247	time-shuffled actions	16/16
Reacher	3074	1.409		1.136	0.263	time-shuffled actions	16/16
Cube	3072	1.171		1.037	0.489	zero actions	14/16
Cube	3073	1.416		1.212	0.500	zero actions	14/16
Cube	3074	1.061		1.008	0.552	time-shuffled actions	14/16

D Cross-Task Threshold Partitions

The 14 rows below are exhaustive: selecting one, two, or three of four tasks as sources creates $4 + 6 + 4$ directional partitions. Each evaluation task retains every training run and all nine checkpoints. The apparent sample size is therefore not inflated by treating checkpoint rows as independent observations.

E Cross-Stressor Pairs and Discordant Cases

Both discordant rows lie near the success-rate criterion: PushT run 3072 under resize and PushT run 3074 under blur each improve success rate by four percentage points, one point below the five-point criterion, while their selective scores increase. The complete table shows these cases rather than only the aggregate classification metrics.

Table 8: Cross-task evaluation of the selective ACPC rule. Thresholds are selected on the source tasks and applied unchanged to all remaining tasks. Metrics first average training runs within each evaluation task and then weight tasks equally. Onset error is measured in Gaussian-augmentation grid units.

Source tasks	Evaluation tasks	τ_R	τ_M	BA	P / R	Onset error mean / max
TwoRoom	PushT, Reacher, Cube	0.3	0.95	0.888	0.884 / 0.981	0.003 / 0.010
PushT	TwoRoom, Reacher, Cube	0.3	0.95	0.894	0.902 / 0.956	0.006 / 0.010
Reacher	TwoRoom, PushT, Cube	0.1	0.95	0.723	0.906 / 0.540	0.029 / 0.050
Cube	TwoRoom, PushT, Reacher	0.3	0.95	0.913	0.950 / 0.938	0.006 / 0.010
TwoRoom, PushT	Reacher, Cube	0.3	0.95	0.874	0.853 / 1.000	0.003 / 0.010
TwoRoom, Reacher	PushT, Cube	0.3	0.95	0.887	0.873 / 0.972	0.003 / 0.010
TwoRoom, Cube	PushT, Reacher	0.3	0.95	0.903	0.925 / 0.972	0.003 / 0.010
PushT, Reacher	TwoRoom, Cube	0.3	0.95	0.896	0.901 / 0.935	0.007 / 0.010
PushT, Cube	TwoRoom, Reacher	0.3	0.95	0.912	0.952 / 0.935	0.007 / 0.010
Reacher, Cube	TwoRoom, PushT	0.3	0.95	0.926	0.972 / 0.907	0.007 / 0.010
TwoRoom, PushT, Reacher	Cube	0.3	0.95	0.858	0.802 / 1.000	0.003 / 0.010
TwoRoom, PushT, Cube	Reacher	0.3	0.95	0.889	0.905 / 1.000	0.003 / 0.010
TwoRoom, Reacher, Cube	PushT	0.3	0.95	0.917	0.944 / 0.944	0.003 / 0.010
PushT, Reacher, Cube	TwoRoom	0.3	0.95	0.935	1.000 / 0.869	0.010 / 0.010

Table 9: Transfer of the calibrated selective score from Gaussian noise to blur and resize. For each task, thresholds are selected on the other three Gaussian-noise tasks and then fixed. The table compares the score change with the change in planning success rate between checkpoints trained with and without Gaussian augmentation.

Visual shift	Pairs	BA	Precision / recall	Spearman	Discordant
All pairs	24	0.889	0.882 / 1.000	0.835	2
Blur	12	0.875	0.889 / 1.000	0.873	1
Resize	12	0.900	0.875 / 1.000	0.746	1

Table 10: All 24 LeWM checkpoint pairs evaluated under blur and resize. ATR_{rel} and SMPR are measured for the checkpoint trained with Gaussian augmentation. ΔP is its change in planning success rate relative to the checkpoint trained without augmentation, and ΔS is the corresponding selective-score change. A success-rate change is positive when $\Delta P \geq 5$ percentage points and the clean success rate decreases by at most five points. Outcome lists the success-rate classification followed by the score classification.

Task	Seed	Shift	ATR_{rel}	SMPR	ΔP	ΔS	Outcome
TwoRoom	3072	blur	0.499	0.885	55.7	1.670	positive / positive
TwoRoom	3072	resize	0.336	0.967	52.3	2.214	positive / positive
TwoRoom	3073	blur	0.781	0.262	36.0	0.729	positive / positive
TwoRoom	3073	resize	0.691	0.361	40.0	1.030	positive / positive
TwoRoom	3074	blur	0.636	0.541	37.7	1.212	positive / positive
TwoRoom	3074	resize	0.617	0.656	34.3	1.277	positive / positive
PushT	3072	blur	0.943	0.939	10.7	0.189	positive / positive
PushT	3072	resize	0.814	0.969	4.0	0.620	nonpositive / positive
PushT	3073	blur	0.846	0.918	7.3	0.515	positive / positive
PushT	3073	resize	0.682	0.969	24.3	1.060	positive / positive
PushT	3074	blur	0.781	0.959	4.0	0.729	nonpositive / positive
PushT	3074	resize	1.173	0.939	-19.7	-0.577	nonpositive / nonpositive
Reacher	3072	blur	0.155	0.990	50.7	2.375	positive / positive
Reacher	3072	resize	0.210	0.990	29.7	2.375	positive / positive
Reacher	3073	blur	0.176	0.990	52.3	2.375	positive / positive
Reacher	3073	resize	0.207	0.990	40.0	2.375	positive / positive
Reacher	3074	blur	0.190	0.990	44.7	2.375	positive / positive
Reacher	3074	resize	0.195	0.990	33.3	2.375	positive / positive
Cube	3072	blur	1.447	0.330	-2.7	-1.488	nonpositive / nonpositive
Cube	3072	resize	1.166	0.530	1.0	-0.552	nonpositive / nonpositive
Cube	3073	blur	1.577	0.260	2.3	-1.923	nonpositive / nonpositive
Cube	3073	resize	1.218	0.560	-1.3	-0.728	nonpositive / nonpositive
Cube	3074	blur	1.220	0.660	-3.0	-0.735	nonpositive / nonpositive
Cube	3074	resize	1.040	0.770	-2.3	-0.134	nonpositive / nonpositive

F Local Gaussian Sensitivity as a Mechanistic Interpretation

For a small isotropic observation perturbation $\delta x \sim \mathcal{N}(0, \sigma^2 I)$, first-order linearization of encoder E followed by rollout map G gives

$$\mathbb{E}[\|\Delta G\|_2^2] \approx \sigma^2 \|J_G J_E\|_F^2. \quad (21)$$

This explains one mechanism by which Gaussian noise training can contract ACPC: it reduces the composed local sensitivity of the encoder–predictor path. It is a local approximation, not a global robustness bound and not a separate contribution.

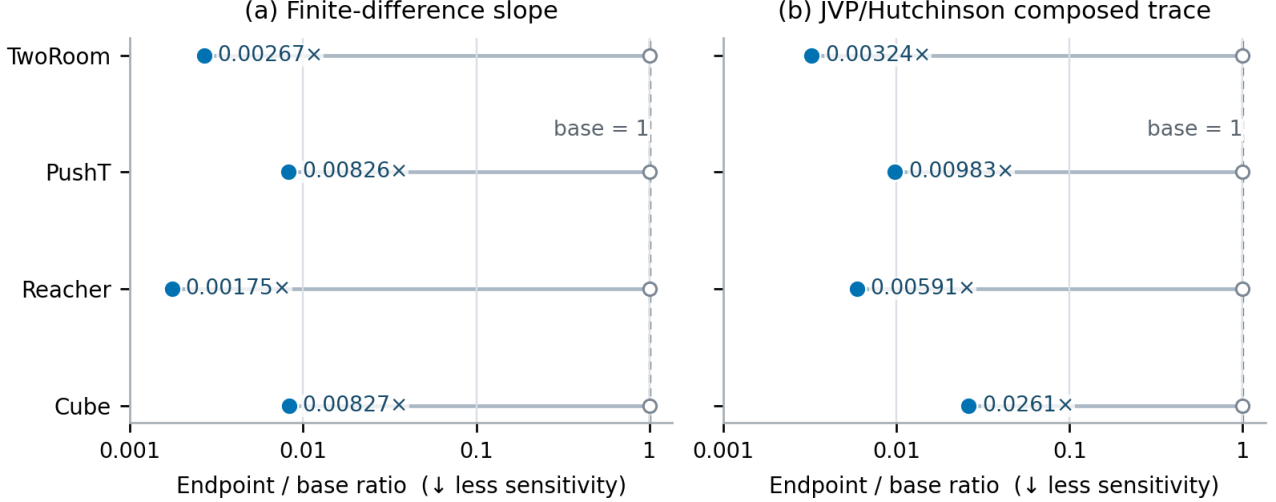


Figure 6: Local-sensitivity ratio of the checkpoint trained with Gaussian augmentation to the checkpoint trained without it, estimated by finite differences and by a JVP/Hutchinson estimate of the composed Jacobian trace. Values below one indicate contraction after augmentation. The two estimators agree in direction for every task; these ratios do not establish task performance.

G Fixed-Pool Candidate Stability

The CEM analysis does not use task-success labels. It compares checkpoints trained without augmentation and with $\sigma_{\max} = 0.08$ on all four tasks. Fixed-pool shards capture the ordered initial proposal, evaluate the nominal and perturbed histories through one shared cost path, and record exact same-winner and elite-set events for 100 histories per task, run, and training condition at severities 0, 0.02, 0.05, 0.08. Adaptive shards use a common initial proposal and common random numbers. Proposal alignment is asserted only while all preceding elite updates remain certified.

For nominal final plan $\mathbf{a}$ and perturbed plan $\tilde{\mathbf{a}}$, the reported positive clean-history regret is

$$[C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a})]_+.$$

It measures decision degradation in the model’s latent goal cost, not an environment outcome. First-action RMS is reported separately. The full-budget evaluation keeps the same definitions but uses 16 fixed histories per task shared across training conditions and runs, $K = 300$, top-30 elites, and 30 CEM steps. Whole-shard runtimes and forward-call counts are retained for provenance, but without matched timing runs that omit the diagnostic they do not estimate incremental overhead.

Across 9,600 joined reduced-budget rows there are zero squared-distance-bound violations, zero false top-1 certificates, and zero false elite certificates. Identity probes have zero five-step displacement and zero first-action RMS; all identity updates remain aligned. These checks establish implementation soundness and are not additional independent statistical samples.

At severity 0.08, the sufficient top-1 certificate covers 10–70% of histories from checkpoints trained with augmentation across task–run blocks (and none from checkpoints trained without augmentation); elite-set coverage is 0–30% with augmentation. Coverage is lower than realized stability because these conditions are sufficient but not necessary.

Table 11: CEM decision statistics at Gaussian perturbation severity 0.08. Values are mean $\pm$ sample SD across independent training runs after histories are averaged within each run. Reduced-budget rows use 100 histories, $K = 64$, top-8 elites, and eight CEM steps; full-budget rows use the same 16 histories per task with $K = 300$, top-30 elites, and 30 steps. Regret is measured with the model’s squared latent goal cost and is not an environment success rate.

Task	Best candidate unchanged		Reduced-budget regret		Full-budget regret	
	No aug.	$\sigma_{\max}=0.08$	No aug.	$\sigma_{\max}=0.08$	No aug.	$\sigma_{\max}=0.08$
TwoRoom	0.13 $\pm$ 0.03	0.93 $\pm$ 0.02	203.81 $\pm$ 9.97	8.47 $\pm$ 2.75	224.97 $\pm$ 30.56	1.75 $\pm$ 1.44
PushT	0.13 $\pm$ 0.09	0.92 $\pm$ 0.01	167.50 $\pm$ 29.53	4.30 $\pm$ 0.28	220.14 $\pm$ 32.97	7.91 $\pm$ 4.23
Reacher	0.07 $\pm$ 0.01	0.96 $\pm$ 0.04	281.12 $\pm$ 20.24	0.47 $\pm$ 0.14	269.44 $\pm$ 22.21	0.28 $\pm$ 0.20
Cube	0.02 $\pm$ 0.02	0.92 $\pm$ 0.03	255.72 $\pm$ 12.28	1.31 $\pm$ 0.11	265.03 $\pm$ 28.08	0.63 $\pm$ 0.22
Equal-task mean	0.09 $\pm$ 0.03	0.93 $\pm$ 0.01	227.04 $\pm$ 16.92	3.64 $\pm$ 0.68	244.89 $\pm$ 27.08	2.65 $\pm$ 1.43

H Qualitative Neighborhood Visualization

The two-dimensional view complements the quantitative analysis by showing how repeated perturbations of the same anchors are arranged before and after noise training. Because t-SNE does not preserve metric geometry, all reported evidence remains in the original projected-rollout space.

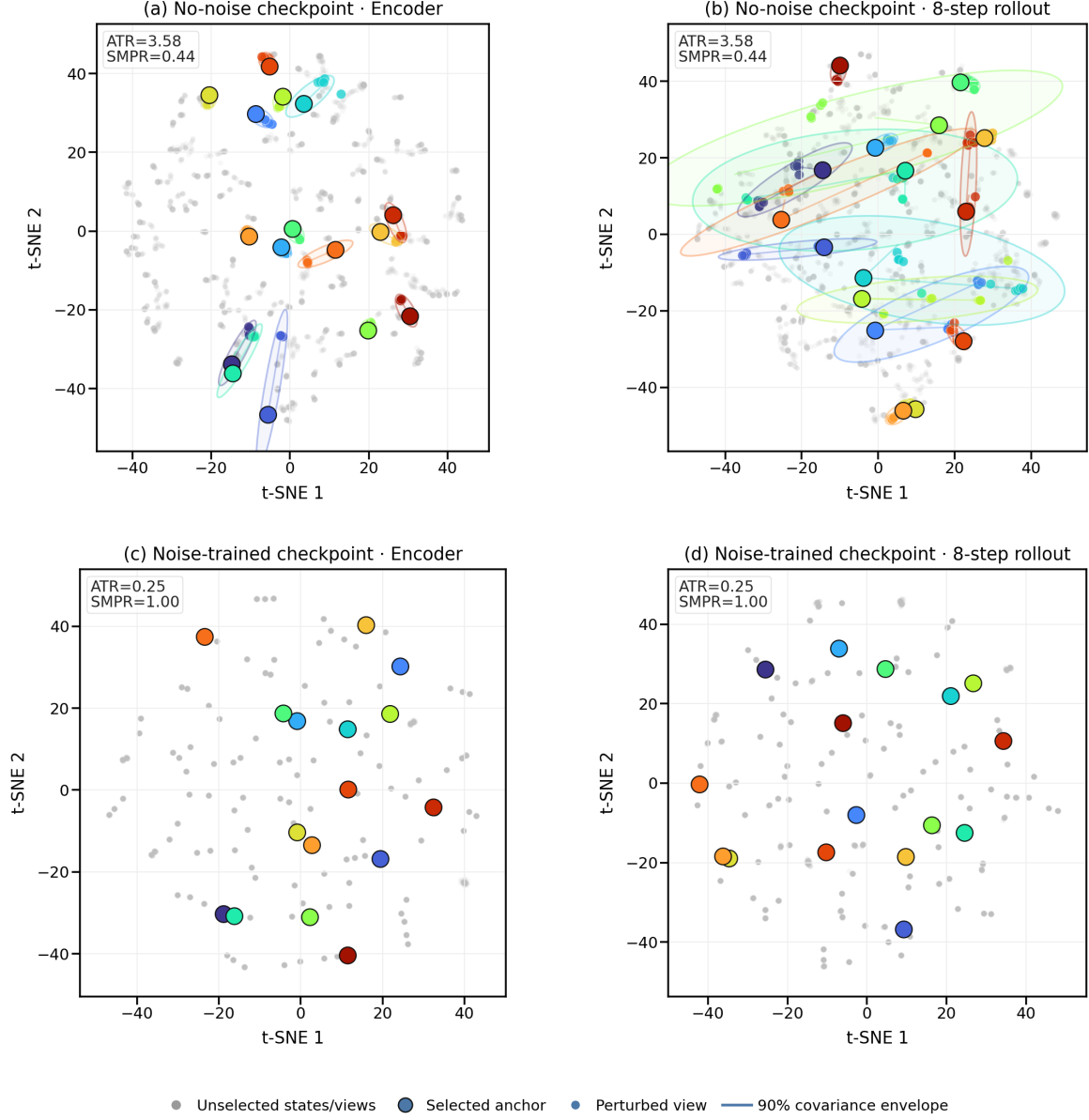


Figure 7: Qualitative PushT neighborhoods for matched perturbations at checkpoints trained without augmentation and with $\sigma_{\max} = 0.08$, shown in encoder and eight-step rollout spaces. t-SNE is not metric preserving: no displayed distance, area, or overlap enters a claim; ATR and SMPR are computed in the original space.